Large Language Models

Week	Task Title	Hands-on Activities	Python Libraries / Tools
1	Text Preprocessing	Clean text, remove special characters, tokenize sentences and words	nltk, re, pandas
2	LLM Tokenization	Explore WordPiece/BPE tokenization and token IDs	transformers, tokenizers
3	Text Generation	Generate text using a pretrained Hugging Face model	transformers, torch
4	Text Embeddings	Generate embeddings and compare sentence similarity	sentence-transformers, numpy
5	Attention Mechanism	Implement and visualize scaled dot-product attention	numpy, torch
6	Prompt Engineering	Experiment with different prompting techniques	transformers
7	Hugging Face Pipelines	Perform summarization, translation, sentiment analysis, and question answering	transformers
8	Local LLM Inference	Run Gemma/Llama locally using Ollama	ollama
9	Chatbot Development	Build a simple chatbot using a local LLM	ollama
10	LangChain Basics	Create prompt templates and simple LLM chains	langchain
11	Vector Embeddings	Create embeddings and store them in ChromaDB	sentence-transformers, chromadb
12	Semantic Document Search	Retrieve relevant documents using vector similarity search	chromadb, langchain
13	PDF Question Answering	Build a chatbot to answer questions from a PDF document	langchain, pypdf, ollama

14	Streamlit LLM App	Create a simple web interface for an LLM application	streamlit, ollama
15	Mini LLM Project	Build a complete LLM application by combining previous concepts	All previous tools

Week 1 – Text Preprocessing

Task Title

Cleaning and Preparing Raw Text for Large Language Models

Objective

- The objective of this task is to introduce students to the first stage of every Natural Language Processing (NLP) and Large Language Model (LLM) pipeline—text preprocessing.
- Students will learn how to convert raw, noisy text collected from websites, social media, PDFs, emails, and user conversations into clean, structured text that can later be tokenized, embedded, and processed by LLMs.
- This cleaned dataset will be reused throughout the remaining weeks.

Task Explanation

Students will develop a Python program to preprocess raw textual data before it is used in a Large Language Model (LLM). The program should read text from a CSV or TXT file, remove punctuation, numbers, special characters, and extra spaces using regular expressions, convert all text to lowercase, and perform sentence and word tokenization using the NLTK library. Finally, the program should display and save the cleaned text for further processing.

Folder Structure

Text_Preprocessing/

```
│
├── dataset/
│   ├── raw_text.csv
│   ├── clean_text.csv
│   └── tokens.csv
│
├── notebook/
│   └── Week1_Text_Preprocessing.ipynb
│
```

├── preprocessing.py

├── README.md

└── requirements.txt

Class Activity	Home Activity
Read a text dataset using Pandas or a text file using Python.	Download any news article, blog, or Wikipedia page (300–500 words).
Clean the text using Python string methods and Regular Expressions (re).	Apply the preprocessing program to the new dataset.
Convert the text to lowercase.	Add stopword removal using NLTK.
Remove punctuation, numbers, and special characters.	Compare the original and processed text.
Perform sentence tokenization using sent_tokenize().	Save the cleaned text as a new .txt or .csv file.
Perform word tokenization using word_tokenize().	Display the total number of sentences and words before and after preprocessing.
Display the processed output in Jupyter Notebook.	Submit the notebook with outputs.

Week 2 – LLM Tokenization

Task Title

Understanding and Implementing LLM Tokenization using WordPiece and BPE

Objective

- To understand how Large Language Models convert text into tokens before processing.
- To explore WordPiece and Byte Pair Encoding (BPE) tokenization techniques.
- To generate token IDs and vocabulary using pretrained Hugging Face tokenizers.
- To prepare tokenized data for text generation in Week 3.

Task Explanation

Large Language Models cannot directly process plain text. Instead, they convert text into smaller units called **tokens**, which are then transformed into numerical IDs before being passed to the model.

In this task, students will use the **clean_text.csv** dataset created in Week 1. They will load pretrained tokenizers such as **BERT (WordPiece)** and **GPT-2 (BPE)** using the Hugging Face Transformers library. Students will compare the tokenization results, analyze token IDs, decode tokens back into text, and observe how different tokenizers split the same sentence. Finally, they will save the generated tokens and vocabulary for use in the next week's text generation task.

Folder Structure

LLM_Tokenization/

```
│
├── dataset/
│   ├── clean_text.csv
│   ├── token_ids.csv
│   ├── vocabulary.txt
│   └── comparison.csv
│
├── notebook/
│   └── Week2_LLM_Tokenization.ipynb
│
├── tokenizer.py
├── README.md
└── requirements.txt
```

Class Activity	**Home Activity**
Load the cleaned dataset (clean_text.csv) generated in Week 1 using Pandas.	Download a new article (300–500 words) from a news website, blog, or Wikipedia.
Install and import the Hugging Face **Transformers** library.	Apply the same tokenization process to the new dataset.
Load the **BERT (WordPiece)** tokenizer.	Compare the tokenization results of BERT and GPT-2 using at least five different sentences.
Tokenize the input text and generate token IDs.	Find the sentence with the highest number of generated tokens and explain why.

Load the **GPT-2 (BPE)** tokenizer.	Decode selected token IDs back into readable text and verify correctness.
Compare the tokenization outputs produced by BERT and GPT-2.	Create a comparison table showing Original Text, BERT Tokens, GPT-2 Tokens, and Token Counts.
Decode token IDs back into the original text.	Save the generated token IDs and vocabulary into CSV and TXT files.
Display vocabulary size and total token count for both tokenizers.	Write a short summary (150–200 words) describing the differences between WordPiece and BPE tokenization.
Save the generated tokens, token IDs, and vocabulary for future tasks.	Submit the notebook, generated files, screenshots, and observations.

Week 3 – Text Generation

Task Title

Generating Text using a Pre-trained Hugging Face Language Model

Objective

- To understand how pre-trained Large Language Models generate human-like text.
- To learn how to use Hugging Face Transformers for text generation.
- To explore the effect of different generation parameters such as max_length, temperature, top_k, and top_p.
- To generate meaningful text using the tokenized input from Week 2.

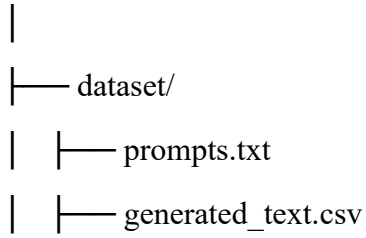
Task Explanation

Large Language Models generate text by predicting the most probable next token based on the input prompt. In this task, students will use a pre-trained Hugging Face language model (such as GPT-2) to generate text from different prompts. They will experiment with various text generation parameters to observe how each parameter affects the quality, creativity, and length of the generated output.

Students will use the **tokenized text and prompts prepared in Week 2** to understand how language models transform input tokens into coherent sentences.

Folder Structure

Text_Generation/



```
│   └── parameter_comparison.csv
│
├── notebook/
│   └── Week3_Text_Generation.ipynb
│
├── text_generation.py
├── README.md
└── requirements.txt
```

Class Activity	Home Activity
Load the GPT-2 pre-trained model using Hugging Face Transformers.	Create five different prompts related to education, healthcare, technology, sports, and business.
Import the tokenizer developed in Week 2.	Generate text for each prompt using different parameter values.
Generate text using a simple prompt.	Compare outputs by changing temperature values (0.5, 0.8, 1.2).
Modify the max_length parameter and observe the output.	Compare outputs using different top_k values (20, 50, 100).
Experiment with different temperature values.	Compare outputs using different top_p values (0.8, 0.9, 0.95).
Experiment with top_k and top_p sampling techniques.	Identify which parameter combination produces the most meaningful text.
Generate multiple outputs for the same prompt.	Save all generated outputs in a CSV file.
Compare deterministic and creative text generation.	Write a short report explaining how each parameter influences text generation.
Display generated outputs in Jupyter Notebook.	Submit the notebook, CSV files, screenshots, and observations.

Week 4 – Text Embeddings

Task Title

Generating and Comparing Text Embeddings using Sentence Transformers

Objective

- To understand the concept of text embeddings and their role in Large Language Models (LLMs).
- To generate sentence embeddings using pre-trained Sentence Transformer models.
- To compare semantic similarity between different sentences using cosine similarity.
- To prepare vector embeddings for semantic search and Retrieval-Augmented Generation (RAG) in upcoming weeks.

Task Explanation

Large Language Models represent text as **high-dimensional vectors**, known as **embeddings**, which capture the semantic meaning of words and sentences. Unlike token IDs, embeddings enable the model to understand contextual relationships between different pieces of text.

In this task, students will use the **Sentence Transformers** library to generate embeddings for the text generated in **Week 3**. They will compute cosine similarity scores between multiple sentences to identify semantically similar content. The generated embeddings will later be stored in a vector database (ChromaDB) during Week 11.

Folder Structure

Text_Embeddings/

```
│
├── dataset/
│   ├── generated_text.csv
│   ├── embeddings.npy
│   ├── similarity_scores.csv
│   └── sentence_pairs.csv
│
├── notebook/
│   └── Week4_Text_Embeddings.ipynb
│
├── embeddings.py
├── README.md
└── requirements.txt
```

Class Activity	Home Activity
Load the generated text dataset from Week 3.	Collect 20 sentences from news articles, blogs, or Wikipedia on different topics.

Install and import the Sentence Transformers library.	Generate embeddings for the collected sentences.
Load a pre-trained embedding model (all-MiniLM-L6-v2).	Compare embeddings between similar and unrelated sentence pairs.
Generate embeddings for each sentence.	Create at least 10 sentence pairs and calculate cosine similarity scores.
Display the embedding vector dimensions.	Identify the top five most similar sentence pairs based on similarity scores.
Calculate cosine similarity between selected sentence pairs.	Explain why certain sentence pairs have high or low similarity scores.
Compare similarity scores for similar and unrelated sentences.	Save the generated embeddings and similarity scores in CSV/NumPy format.
Store embeddings as a NumPy file (embeddings.npy).	Submit the notebook, output files, screenshots, and observations.
Display the similarity matrix in Jupyter Notebook.	Write a short report explaining the importance of embeddings in semantic search and LLM applications.

Week 5 – Attention Mechanism

Task Title

Implementing and Visualizing Scaled Dot-Product Attention

Objective

- To understand the working principle of the Scaled Dot-Product Attention mechanism used in Transformer models.
- To implement the attention mechanism from scratch using NumPy and PyTorch.
- To visualize attention weights and understand how a model focuses on important words in a sentence.
- To prepare for Prompt Engineering and advanced LLM concepts in the following weeks.

Task Explanation

The **Attention Mechanism** is the core component of Transformer-based Large Language Models. Instead of processing words one after another like traditional RNNs, Transformers use attention to determine which words in a sentence are most relevant to each other.

In this task, students will manually implement the **Scaled Dot-Product Attention** algorithm. They will create **Query (Q)**, **Key (K)**, and **Value (V)** matrices, calculate attention scores, apply the Softmax function

to obtain attention weights, and generate the final attention output. Students will also visualize the attention matrix to understand how the model distributes focus across different words.

This activity helps students understand the internal working of Transformer models before using advanced LLM frameworks.

Folder Structure

Attention_Mechanism/

```
│
├── dataset/
│   ├── sample_sentences.txt
│   ├── attention_weights.csv
│   ├── attention_output.csv
│   └── attention_heatmap.png
│
├── notebook/
│   └── Week5_Attention_Mechanism.ipynb
│
├── attention.py
├── README.md
└── requirements.txt
```

Class Activity	Home Activity
Load sample sentences or embeddings generated in Week 4.	Select five different sentences with varying lengths.
Create Query (Q), Key (K), and Value (V) matrices.	Apply the attention implementation to all selected sentences.
Calculate attention scores using the Scaled Dot-Product Attention formula.	Compare attention weights for short and long sentences.
Apply the Softmax function to normalize attention scores.	Analyze which words receive the highest attention and explain why.
Compute the final attention output matrix.	Experiment by changing the embedding dimension and observe the changes in attention scores.

Display Query, Key, Value, Attention Scores, and Attention Weights.	Save the generated attention weights and outputs as CSV files.
Visualize the attention weights using a heatmap.	Capture screenshots of the heatmap and include observations in the notebook.
Execute the complete implementation in Jupyter Notebook.	Submit the notebook, Python script, generated files, and screenshots.
Compare manually implemented attention with PyTorch implementation.	Write a short report explaining the importance of attention in Transformer-based LLMs.

Week 6 – Prompt Engineering

Task Title

Designing Effective Prompts for Large Language Models

Objective

- To understand the importance of prompt engineering in improving LLM responses.
- To experiment with different prompting techniques such as Zero-shot, One-shot, Few-shot, Role Prompting, and Chain-of-Thought (CoT).
- To compare the quality of responses generated using different prompt styles.
- To create reusable prompt templates for real-world LLM applications.

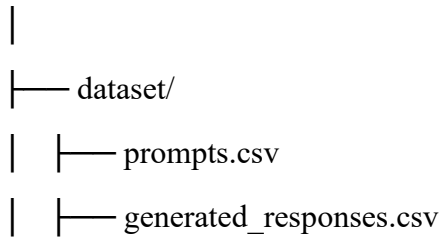
Task Explanation

Prompt Engineering is the process of designing clear and effective instructions that guide a Large Language Model to generate accurate and relevant responses. The quality of an LLM's output depends significantly on how the prompt is written.

In this task, students will use a pre-trained language model from Hugging Face and experiment with various prompting techniques. They will observe how changing the prompt structure affects the generated response. Students will compare outputs, evaluate response quality, and develop reusable prompt templates that can be integrated into future chatbot and document question-answering applications

Folder Structure

Prompt_Engineering/



```
│   ├── prompt_comparison.csv
│   └── evaluation_results.csv
│
├── notebook/
│   └── Week6_Prompt_Engineering.ipynb
│
├── prompt_engineering.py
├── README.md
└── requirements.txt
```

Class Activity	Home Activity
Load a pre-trained language model using Hugging Face Transformers.	Create five real-world problems from education, healthcare, finance, customer support, and programming.
Create a simple **Zero-shot** prompt and generate a response.	Solve each problem using Zero-shot prompting.
Design a **One-shot** prompt by providing one example.	Design One-shot prompts for all five problems and compare the responses.
Create a **Few-shot** prompt using multiple examples.	Generate responses using Few-shot prompting and evaluate improvements.
Assign a role using **Role Prompting** (e.g., "You are a software engineer").	Experiment with different roles (Teacher, Doctor, HR Manager, AI Engineer, Travel Guide) and compare outputs.
Apply **Chain-of-Thought (CoT)** prompting for reasoning-based questions.	Create three reasoning questions and compare normal prompting with Chain-of-Thought prompting.
Compare responses generated by each prompting technique.	Develop three reusable prompt templates for different applications.
Record response quality, accuracy, and completeness in a comparison table.	Save all prompts, responses, and evaluation results in CSV format and submit the notebook.

Week 7 – Hugging Face Pipelines

Task Title

Performing NLP Tasks using Hugging Face Pipelines

Objective

- To understand how Hugging Face Pipelines simplify the use of pre-trained transformer models.

- To perform multiple NLP tasks such as text summarization, translation, sentiment analysis, and question answering.
- To compare the outputs of different pipeline tasks using real-world text.
- To prepare reusable NLP components for chatbot and LLM applications.

Task Explanation

Hugging Face Pipelines provide a simple interface to use pre-trained transformer models without manually loading tokenizers and models. Instead of writing separate code for every NLP task, pipelines allow developers to perform tasks like **text summarization, translation, sentiment analysis, text classification, and question answering** with just a few lines of code.

In this task, students will use the prompts and text generated in the previous weeks to perform different NLP tasks using Hugging Face Pipelines. They will compare outputs, evaluate model performance, and save the results for future integration into chatbot applications.

Folder Structure

HuggingFace_Pipelines/

```
│
├── dataset/
│   ├── input_text.csv
│   ├── summarized_text.csv
│   ├── translated_text.csv
│   ├── sentiment_results.csv
│   ├── qa_results.csv
│   └── pipeline_comparison.csv
│
├── notebook/
│   └── Week7_HuggingFace_Pipelines.ipynb
│
├── pipelines.py
├── README.md
└── requirements.txt
```

Class Activity	Home Activity

Install and import the Hugging Face Transformers library.	Download a news article or blog (500–800 words) for testing.
Load the **Sentiment Analysis** pipeline and classify sample text.	Perform sentiment analysis on the downloaded article and summarize the findings.
Load the **Summarization** pipeline and generate a summary of a long paragraph.	Compare summaries generated using different max_length values.
Load the **Translation** pipeline and translate English text into another language.	Translate the same text into two different languages and compare the outputs.
Load the **Question Answering** pipeline and answer questions from a given context.	Create five questions from the article and evaluate the model's answers.
Compare outputs from different pipelines.	Create a comparison table for Original Text, Summary, Translation, Sentiment, and Answers.
Save all outputs into CSV files.	Save the generated outputs and screenshots in the project folder.
Execute the notebook and verify all pipeline outputs.	Submit the notebook, generated files, screenshots, and observations.

Week 8 – Local LLM Inference

Task Title

Running Large Language Models Locally using Ollama

Objective

- To understand the advantages of running Large Language Models (LLMs) locally.
- To install and configure Ollama for local LLM inference.
- To download and interact with open-source LLMs such as **Gemma** or **Llama 3**.
- To compare responses generated by local LLMs using different prompts.
- To prepare a local LLM environment for chatbot development in Week 9.

Task Explanation

Most Large Language Models are accessed through cloud APIs. However, many open-source LLMs can now be executed locally using tools such as **Ollama**. Running LLMs locally provides benefits such as improved privacy, offline access, lower API costs, and greater flexibility for application development.

In this task, students will install **Ollama**, download a local LLM (such as **Gemma 3** or **Llama 3**), and interact with the model using Python. They will send different prompts to the model, observe the generated responses, compare outputs for various prompt styles, and save the conversations for future chatbot development.

Folder Structure

```
Local_LLM_Inference/
│
├── dataset/
│   ├── prompts.csv
│   ├── llm_responses.csv
│   ├── response_comparison.csv
│   └── conversation_history.txt
│
├── notebook/
│   └── Week8_Local_LLM_Inference.ipynb
│
├── local_llm.py
├── README.md
└── requirements.txt
```

Class Activity	Home Activity
Install Ollama on the local machine.	Install another supported local LLM (Gemma or Llama) and compare its responses with the model used in class.
Verify the Ollama installation using terminal commands.	Create ten prompts from different domains such as education, healthcare, coding, travel, and business.
Download a local LLM (Gemma 3 or Llama 3).	Generate responses for all prompts and save them in a CSV file.
Import the Ollama Python library and connect to the local model.	Compare the quality, response length, and accuracy of the generated outputs.
Generate responses for simple text prompts.	Experiment with prompts of different complexity and observe the differences in responses.
Modify prompts and observe changes in generated output.	Create a prompt that produces an incorrect response and rewrite it to improve the answer.
Save prompts and responses in a CSV file.	Record observations on how prompt wording affects response quality.

Display all responses in Jupyter Notebook.	Submit the notebook, response files, screenshots, and observations.

Week 9 – Chatbot Development

Task Title

Building an Interactive AI Chatbot using Ollama

Objective

- To understand the architecture of an AI chatbot powered by a local Large Language Model (LLM).
- To build a command-line chatbot using Ollama and Python.
- To implement conversation history for context-aware responses.
- To prepare the chatbot for integration with LangChain in the next week.

Task Explanation

Chatbots are one of the most common real-world applications of Large Language Models. Instead of answering a single prompt, chatbots maintain an interactive conversation with users by processing multiple queries and generating contextually relevant responses.

In this task, students will build a chatbot using **Python** and **Ollama**. The chatbot will continuously accept user input, send prompts to a locally running LLM, receive responses, and display them in the terminal. Students will also implement conversation history so that the chatbot can respond based on previous interactions. Finally, the conversation logs will be saved for future integration with LangChain.

Folder Structure

Chatbot_Development/

```
│
├── dataset/
│   ├── chat_history.csv
│   ├── sample_questions.csv
│   ├── chatbot_logs.txt
│   └── responses.csv
│
├── notebook/
│   └── Week9_Chatbot_Development.ipynb
│
├── chatbot.py
```

├── README.md

└── requirements.txt

Class Activity	Home Activity
Install and configure Ollama with a local LLM (Gemma or Llama).	Create a chatbot for a specific domain such as Education, Healthcare, Banking, Travel, or Customer Support.
Create a Python program to accept user input continuously.	Add at least 20 sample user questions related to the selected domain.
Send user prompts to the local LLM using the Ollama Python library.	Test the chatbot using different types of questions (factual, reasoning, coding, and general conversation).
Display the model's responses in the terminal.	Identify incorrect or incomplete responses and improve the prompts.
Implement a conversation loop until the user types **exit**.	Save all conversations in a CSV file.
Store the conversation history in a list or file.	Analyze the chatbot's responses and document its strengths and limitations.
Save user questions and chatbot responses in **chat_history.csv**.	Add timestamps to each conversation record.
Execute the chatbot and verify that multiple conversations are stored correctly.	Submit the notebook, chatbot source code, conversation logs, screenshots, and observations.

Week 10 – LangChain Basics

Task Title

Building LLM Applications using LangChain Prompt Templates and Chains

Objective

- To understand the fundamentals of the LangChain framework for developing LLM applications.
- To create reusable prompt templates for different user queries.
- To build simple LLM chains that process user input and generate responses.
- To integrate the chatbot developed in Week 9 with LangChain.
- To prepare the chatbot for vector database integration in the following weeks.

Task Explanation

As LLM applications become more complex, managing prompts and model interactions using plain Python code becomes difficult. **LangChain** is a framework that helps developers build modular and

scalable AI applications by organizing prompts, LLM calls, and workflows into reusable components called **chains**.

In this task, students will convert the chatbot developed in **Week 9** into a LangChain-based application. They will create prompt templates with placeholders, pass dynamic user inputs, execute LLM chains, and compare the LangChain implementation with the previous chatbot. This modular design will serve as the foundation for Retrieval-Augmented Generation (RAG) in the upcoming weeks.

Folder Structure

LangChain_Basics/

```
│
├── dataset/
│   ├── prompts.csv
│   ├── user_queries.csv
│   ├── chatbot_responses.csv
│   └── chain_results.csv
│
├── notebook/
│   └── Week10_LangChain_Basics.ipynb
│
├── langchain_chatbot.py
├── README.md
└── requirements.txt
```

Class Activity	Home Activity
Install LangChain and required dependencies.	Create a domain-specific chatbot (College Assistant, Library Assistant, Travel Guide, or Customer Support).
Connect LangChain with the local Ollama model.	Design five different prompt templates for different user scenarios.
Create a Prompt Template with placeholders for user input.	Test the chatbot using at least 20 different user queries.
Pass dynamic user input to the prompt template.	Modify the prompt template to improve response quality and compare the outputs.

Build a simple LLM Chain using LangChain.	Create two additional chains for different tasks (e.g., summarization and question answering).
Execute the chain and generate responses.	Compare the chatbot developed in Week 9 with the LangChain version and document the differences.
Save user queries and generated responses into a CSV file.	Save all prompt templates and generated outputs in the project folder.
Display the chatbot responses in Jupyter Notebook.	Submit the notebook, source code, screenshots, and observations.

Week 11 – Vector Embeddings with ChromaDB

Task Title

Creating and Storing Vector Embeddings using ChromaDB

Objective

- To understand the role of vector databases in Large Language Model (LLM) applications.
- To generate vector embeddings for documents using Sentence Transformers.
- To store embeddings in ChromaDB for efficient semantic retrieval.
- To perform similarity searches using vector embeddings.
- To prepare the knowledge base for Retrieval-Augmented Generation (RAG) in Week 12.

Task Explanation

Large Language Models have limited knowledge based on their training data. To enable LLMs to answer questions from **custom documents**, the documents must first be converted into **vector embeddings** and stored in a **vector database**.

In this task, students will use the **Sentence Transformers** library to generate embeddings for text documents and store them in **ChromaDB**. They will perform similarity searches by providing user queries and retrieving the most relevant documents based on semantic meaning rather than exact keyword matching.

The vector database created in this week will be used as the knowledge base for building a semantic search application in Week 12.

Folder Structure

Vector_Embeddings/

│

├── dataset/

```
│   ├── documents/
│   │   ├── document1.txt
│   │   ├── document2.txt
│   │   └── document3.txt
│   │
│   ├── queries.csv
│   ├── retrieved_results.csv
│   └── embeddings.npy
│
├── chroma_db/
│
├── notebook/
│   └── Week11_Vector_Embeddings.ipynb
│
├── vector_store.py
├── README.md
└── requirements.txt
```

Class Activity	**Home Activity**
Install ChromaDB and Sentence Transformers libraries.	Download or create at least **10 text documents** related to a single domain (Education, Healthcare, AI, Sports, etc.).
Load sample text documents from the dataset folder.	Generate embeddings for all collected documents.
Generate embeddings using the Sentence Transformer model (all-MiniLM-L6-v2).	Store the generated embeddings in ChromaDB.
Create a ChromaDB collection.	Add metadata such as document title, category, and source to each document.
Store document embeddings in ChromaDB.	Create at least **10 user queries** for testing.
Perform similarity search for sample queries.	Retrieve the **Top-5** most relevant documents for each query.

Display retrieved documents and similarity results.	Compare retrieved results with keyword-based search and document your observations.
Save retrieval results in a CSV file.	Submit the notebook, database files, screenshots, and observations.

Week 12 – Semantic Document Search

Task Title

Building a Semantic Document Search System using LangChain and ChromaDB

Objective

- To understand the concept of semantic search using vector embeddings.
- To retrieve relevant documents based on semantic similarity instead of keyword matching.
- To integrate LangChain with ChromaDB for document retrieval.
- To build a semantic search application that serves as the retrieval component of a Retrieval-Augmented Generation (RAG) system.
- To prepare the retrieved documents for LLM-based question answering in Week 13.

Task Explanation

Traditional search engines retrieve documents by matching keywords, which often produces irrelevant results when different words express the same meaning. **Semantic Search** overcomes this limitation by comparing the vector embeddings of documents and user queries.

In this task, students will use the **ChromaDB vector database** created in **Week 11** along with **LangChain** to build a semantic document search application. When a user enters a natural language query, the system will generate an embedding for the query, search the vector database, retrieve the most relevant documents, and display the results ranked by semantic similarity.

This retrieval system will be extended into a **PDF Question Answering (RAG)** application in Week 13.

Folder Structure

Semantic_Document_Search/

```
│
├── dataset/
│   ├── documents/
│   ├── queries.csv
│   ├── search_results.csv
│   └── similarity_scores.csv
```

```
│
├── chroma_db/
│
├── notebook/
│   └── Week12_Semantic_Document_Search.ipynb
│
├── semantic_search.py
├── README.md
└── requirements.txt
```

Class Activity	Home Activity
Load the ChromaDB database created in Week 11.	Add at least **10 new documents** to the existing ChromaDB collection.
Connect LangChain with ChromaDB.	Create **15 user queries** from different topics related to the stored documents.
Load the Sentence Transformer embedding model.	Perform semantic searches for all user queries.
Accept a user query from the keyboard.	Compare semantic search results with keyword-based search and record the differences.
Convert the query into a vector embedding.	Retrieve the **Top-5** relevant documents for every query.
Retrieve the Top-5 most relevant documents using similarity search.	Calculate and record similarity scores for each retrieved document.
Display retrieved documents along with similarity scores.	Store all search results and similarity scores in CSV format.
Save search results into a CSV file.	Write a short report explaining how semantic search improves information retrieval compared to traditional keyword search.
Execute the complete semantic search application in Jupyter Notebook.	Submit the notebook, source code, screenshots, and observations.

Week 13 – PDF Question Answering

Task Title

Building a PDF Question Answering System using LangChain, ChromaDB, and Ollama

Objective

- To understand the concept of Retrieval-Augmented Generation (RAG).
- To extract text from PDF documents and convert it into vector embeddings.
- To retrieve relevant information from PDF documents using ChromaDB.
- To generate context-aware answers using a local Large Language Model (LLM) through Ollama.
- To prepare the backend for a web-based LLM application in Week 14.

Task Explanation

Organizations often store information in PDF documents such as manuals, policies, reports, and research papers. Instead of manually reading these documents, a **Retrieval-Augmented Generation (RAG)** system enables users to ask questions in natural language and receive accurate answers based on the document content.

In this task, students will build a **PDF Question Answering System**. They will extract text from a PDF file, split the text into smaller chunks, generate embeddings, store them in ChromaDB, retrieve the most relevant chunks for a user query, and use **Ollama** with a local LLM to generate answers based only on the retrieved content.

Folder Structure

PDF_Question_Answering/

```
│
├── dataset/
│   ├── sample.pdf
│   ├── extracted_text.txt
│   ├── chunks.csv
│   ├── queries.csv
│   └── answers.csv
│
├── chroma_db/
│
├── notebook/
│   └── Week13_PDF_Question_Answering.ipynb
│
├── pdf_qa.py
```

├── README.md

└── requirements.txt

Class Activity	Home Activity
Install LangChain, ChromaDB, Ollama, and PyPDF libraries.	Download a PDF (10–20 pages) related to AI, Machine Learning, or any technical topic.
Load and extract text from a sample PDF document.	Extract the text from the selected PDF.
Split the extracted text into smaller chunks.	Experiment with different chunk sizes and overlap values.
Generate embeddings for each text chunk using Sentence Transformers.	Create a ChromaDB collection for the new PDF.
Store the embeddings in ChromaDB.	Prepare at least **15 user questions** based on the PDF content.
Accept a user query and retrieve the most relevant text chunks.	Retrieve the Top-5 relevant chunks for every question.
Pass the retrieved context to the local LLM using Ollama.	Compare answers generated with and without retrieval and record the differences.
Display the generated answer along with the retrieved context.	Save all questions, retrieved chunks, and answers into CSV files.
Execute the complete PDF Question Answering system in Jupyter Notebook.	Submit the notebook, source code, screenshots, and observations.

Week 14 – Streamlit LLM Application

Task Title

Developing an Interactive LLM Web Application using Streamlit and Ollama

Objective

- To understand how to deploy an LLM application using Streamlit.
- To develop a user-friendly web interface for the PDF Question Answering system built in Week 13.
- To integrate Streamlit with LangChain, ChromaDB, and Ollama.
- To enable users to upload PDF documents, ask questions, and receive AI-generated responses through a web browser.
- To prepare the complete application for the final capstone project in Week 15.

Task Explanation

Most AI applications are deployed with graphical user interfaces so that end users can interact with them easily. **Streamlit** is a Python framework that enables developers to quickly build web applications for machine learning and LLM projects.

In this task, students will convert the **PDF Question Answering System** developed in **Week 13** into an interactive web application. Users will be able to upload a PDF document, ask questions, retrieve relevant document sections, and receive AI-generated answers through a browser. Students will also enhance the interface with a sidebar, chat history, loading indicators, and response formatting.

Folder Structure

```
Streamlit_LLM_App/
│
├── dataset/
│   ├── sample.pdf
│   ├── queries.csv
│   ├── answers.csv
│   └── uploaded_files/
│
├── chroma_db/
│
├── app.py
├── pdf_qa.py
├── README.md
├── requirements.txt
└── assets/
    ├── logo.png
    └── screenshots/
```

Class Activity	Home Activity
Install Streamlit and required Python libraries.	Improve the user interface by adding a custom title, logo, and sidebar.
Create a basic Streamlit application.	Display the uploaded PDF filename and total number of pages.

Add a PDF file uploader using Streamlit.	Add validation to allow only PDF files.
Load the uploaded PDF and extract its text.	Allow users to upload and switch between multiple PDF documents.
Connect the application to the ChromaDB database.	Add a "Clear Chat History" button.
Accept user questions through a text input box.	Display the conversation history on the web page.
Retrieve relevant document chunks using semantic search.	Show the retrieved document chunks below each generated answer.
Generate answers using Ollama and display them on the web interface.	Display the response generation time for each query.
Test the application with multiple user queries.	Package the application with a professional README and screenshots.
Run the application using streamlit run app.py.	Submit the complete project folder with source code and screenshots.

Week 15 – Mini LLM Capstone Project

Task Title

Developing an End-to-End Large Language Model (LLM) Application

Objective

- To integrate all the concepts learned throughout the course into a complete LLM application.
- To develop a real-world AI solution using LangChain, Ollama, ChromaDB, and Streamlit.
- To implement Retrieval-Augmented Generation (RAG) for answering questions from custom documents.
- To deploy a fully functional web-based LLM application.
- To demonstrate software development, problem-solving, and documentation skills through a capstone project.

Task Explanation

Throughout the previous fourteen weeks, students have learned every major component required to build an LLM application—from text preprocessing and tokenization to embeddings, prompt engineering, semantic search, Retrieval-Augmented Generation (RAG), and web deployment.

In this final project, students will integrate all these components into a **complete end-to-end LLM application**. The application should allow users to upload documents, retrieve relevant information using semantic search, generate context-aware answers using a local LLM, and interact with the system through a Streamlit web interface.

Students may choose any domain such as **Education, Healthcare, Finance, Legal, Human Resources, Agriculture, Tourism, Customer Support, or Technical Documentation**.

Folder Structure

Mini_LLM_Project/

```
│
├── app.py
├── preprocessing.py
├── embedding.py
├── rag_pipeline.py
├── chatbot.py
├── requirements.txt
├── README.md
│
├── dataset/
│   ├── documents/
│   ├── uploaded_files/
│   ├── queries.csv
│   ├── answers.csv
│   └── chat_history.csv
│
├── chroma_db/
│
├── assets/
│   ├── screenshots/
│   └── logo.png
│
└── notebook/
    └── Week15_Mini_LLM_Project.ipynb
```

Class Activity	Home Activity

Select a real-world problem domain for the LLM application.	Improve the application by adding additional features such as dark mode, downloadable chat history, or multiple document upload.
Prepare and preprocess the dataset collected for the selected domain.	Test the application with at least **20 different user questions** and evaluate the responses.
Generate document embeddings and store them in ChromaDB.	Optimize prompt templates to improve response quality.
Build the Retrieval-Augmented Generation (RAG) pipeline using LangChain.	Compare the application's responses with another LLM and document the observations.
Connect the RAG pipeline to a local LLM using Ollama.	Record the application's strengths, limitations, and possible future improvements.
Develop a Streamlit-based user interface for the application.	Create a demonstration video (3–5 minutes) showing the complete workflow.
Allow users to upload documents and ask questions through the web application.	Prepare a presentation explaining the project architecture, implementation, and results.
Test the complete application using multiple user queries.	Upload the complete source code to GitHub with proper documentation.
Document the project with installation steps and user instructions.	Submit the project source code, report, presentation, and demonstration video.